Notes on Reinforcement Learning

Hyoung Joon, Kim

June 13, 2026

Contents

1 Multi-armed Bandits **4**

1.1 Introduction . . . 4

1.2 k-armed Bandit Problem . . . 4

1.3 Balancing between Exploring and Exploiting . . . 4

1.3.1 Action-Value Methods . . . 4

1.3.1.1 Estimation of Value . . . 5

1.3.1.1.1 Methods for Stationary Bandit Problem . . . 5

1.3.1.1.2 Methods for Non-stationary Bandit Problem . . . 5

1.3.1.2 Selection of Actions . . . 6

1.3.1.2.1 Greedy Action Selection . . . 6

1.3.1.2.2 ε-greedy Selection . . . 6

1.3.1.2.3 Upper-Confidence-Bound Action Selection . . . 7

1.3.1.3 Optimistic Initial Values . . . 7

1.3.2 Gradient Bandit Algorithm . . . 8

1.3.2.1 Algorithm . . . 8

2 Markov Decision Processes **10**

2.1 Introduction . . . 10

2.2 Finite Markov Decision Process . . . 10

2.2.1 Definition . . . 10

2.2.2 Markov Property . . . 11

2.3 Episodic Task and Continuing Task . . . 11

2.4 Policies and Value Functions . . . 12

2.4.1 Definitions . . . 12

2.4.2 Bellman Equation . . . 13
2.5 Optimal Policies and Optimal Value Functions . . . 14
2.5.1 Definition . . . 14
2.5.2 Bellman Optimality Equation . . . 14

3 Dynamic Programming **16**
3.1 Introduction . . . 16
3.2 Policy Evaluation . . . 16
3.3 Policy Improvement . . . 17
3.4 Obtaining Optimal Policy . . . 18
3.4.1 Policy Iteration . . . 18
3.4.2 Value Iteration . . . 19
3.5 Asynchronous Dynammic Programming . . . 20
3.6 Generalized Policy Iteration . . . 20

4 Monte Carlo Methods **21**
4.1 Introduction . . . 21
4.2 Monte Carlo Prediction . . . 21
4.3 Monte Carlo Estimation of Action Values . . . 22
4.4 Monte Carlo Control . . . 22
4.4.1 Policy Improvement . . . 23
4.4.2 Algorithms . . . 23
4.4.2.1 Monte Carlo Control with Exploring Start . . . 23
4.4.2.2 Monte Carlo Control without Exploring Start . . . 24
4.4.3 Dealing with Infinite Iterations . . . 25
4.5 Off-policy Monte Carlo Method . . . 25
4.5.1 Introduction . . . 25
4.5.2 Importance Sampling . . . 26
4.5.3 Off-policy Monte Carlo Prediction . . . 26
4.5.3.1 Incremental Implementation . . . 27
4.5.4 Off-policy Monte Carlo Control . . . 28
4.6 Reducing Variance of Estimators . . . 28

4.6.1 Discount-aware Importance Sampling 28
4.6.2 Per-decision Importance Sampling 29

Chapter 1

Multi-armed Bandits

1.1 Introduction

In this chapter, we discuss about the evaluative aspect of reinforcement learning in simplified setting. We consider situation with only one *state*, with multiple choices. This can be understood as k-armed bandit problem.

1.2 k-armed Bandit Problem

In k-armed bandit problems, we can pick one of the k actions at time step t, and recieve the reward and then determine the action of next time step $t+1$, using the reward. Our objective is to maximize the total expected reward over some time period. Following are some notations and definitions:

- Action selected on time step t: A_t;
- Reward given on time step t: R_t;
- The expected reward of selected action a (called value): $q_*(a) = \mathbb{E}[R_t|A_t = a]$

If the values of actions are known exactly, we just have to select the action which has largest value at each time step. So we assume that the exact values of actions are not known, and this is the usual situation, too. Since we do not know exact values, we try to estimate the values.

1.3 Balancing between Exploring and Exploiting

1.3.1 Action-Value Methods

Since we assume that the exact value of each actions are not known, we try to estimate the value of actions, and use it to make action selection decisions. This is collectively called as **action-value methods**.

1.3.1.1 Estimation of Value

1.3.1.1.1 Methods for Stationary Bandit Problem The reward probabilties does not change over time in *stationary problems*. The following is a natural way of estimating the value of actions.

Definition 1.3.1.1 (Sample-average Method)

Let A_i and R_i be the action selected at time step i and the corresponding reward. Then, the **sample-average** method estimates the value of action a at time t as following:

$$Q_t(a) = \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

It is guaranteed to converge to true value, $q_*(a)$, as the time step increases. This can be verified with the law of large numbers.

Considering computational efficiencies, recording all the rewards of each actions selected each time is not memory-efficient. We can use incremental method to implement the sample-average method efficiently.

Algorithm 1: Incremental Sample Average Algorithm

Input : t ..... current time step
a ..... selected action in time step t
n ..... the times of action a was selected until time step t (excluding current selection)
R_n ..... reward given to n-th selected action a
$Q_t(a)$ ..... estimation of value of action a in current time step t

Output: $Q_{t+1}(a)$ ..... new estimate of value of action a

$Q_{t+1}(a) \leftarrow Q_t(a) + \frac{1}{n}(R_n - Q_t(a))$
return $Q_{t+1}(a)$

1.3.1.1.2 Methods for Non-stationary Bandit Problem The averaging methods discussed above are not appropriate for *non-stationary* bandit problem, whose reward probabilites changes over time. For non-stationary bandit problem, we give more weight to recent rewards of action than to long-past rewards. We can implement this via using the moving average.

Algorithm 2: Weighted Average Method Algorithm

Input : t ..... current time step
a ..... selected action in time step t
n ..... the times of action a was selected until time step t (excluding current selection)
R_n ..... reward given to n-th selected action a
$Q_t(a)$ ..... estimation of value of action a in current time step t
$\alpha \in (0, 1]$ ..... constant step size

Output: $Q_{t+1}(a)$ ..... new estimate of value of action a

$Q_{t+1}(a) \leftarrow Q_t(a) + \alpha(R_n - Q_t(a))$
return $Q_{t+1}(a)$

We can verify that this algorithm weakens the effect of rewards given earlier. We first assume that $Q_n(a)$ is the value of

action a at the time step when the action a is selected for n times.

$$\begin{aligned}
Q_{n+1}(a) &= Q_n(a) + \alpha(R_n - Q_n(a)) \\
&= \alpha R_n + (1-\alpha)Q_n(a) \\
&= \alpha R_n + (1-\alpha)(\alpha R_{n-1} + (1-\alpha)Q_{n-1}(a)) = \alpha R_n + (1-\alpha)\alpha R_n + (1-\alpha)^2 Q_n(a) \\
&= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 \alpha R_{n-2} + \cdots + (1-\alpha)^{n-1}\alpha R_1 + (1-\alpha)^n Q_1(a) \\
&= (1-\alpha)^n Q_1(a) + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i
\end{aligned}$$

As one can see, the earlier rewards decreases exponentially.

The step size α can vary from step to step. Then, the conditions required for $Q_t(a)$ to converge to true action value with probability 1 is given as:

Theorem 1.3.1.1 (Conditions for Convergence)

Let $\alpha_n(a)$ be the step-size parameter used to process the reward received after the n-th selection of action a. Then, $Q_t(a)$ converges to $q_*(a)$ as $t \to \infty$ with probability 1 when following conditions are satisfied:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \text{ and } \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

But these varying step size is seldom used in applications and empirical search, since it often converge slowly or need considerable tuing in order to obtain a satisfactory convergence rate.

1.3.1.2 Selection of Actions

1.3.1.2.1 Greedy Action Selection The simplest way of action selection is to select the action with the largest value, which is called the *greedy action*.

Algorithm 3: Greedy Action Selection Algorithm

Input : $\mathcal{A}$ ..... set of all possible actions
t ..... current time step
$Q_t(a)$ ..... estimated value of action a at time step t

Output: A_t ..... action selection at time t

$A_t \leftarrow \arg\max_a Q_t(a)$
return A_t

1.3.1.2.2 ε-greedy Selection Since what we can know is only the estimate of the value in current time step, the greedy action may not be the best action choice in the long run. Selecting the action other than greedy action is called **exploring**. One of the simple method of exploration is to select non-greedy actions with certain probability ε.

Algorithm 4: ε-greedy Selection

Input : $\mathcal{A}$ ..... set of all possible actions
t ..... time step
$Q_t(a)$ ..... estimation of value of action a on time step t
$\varepsilon \in [0, 1]$ ..... probability

Output: A_t ..... action selection at time t

$p \sim \mathcal{U}(0, 1)$
if $p > \varepsilon$ **then**
  $A_t \leftarrow \arg\max_a Q_t(a)$
else
  $A_t \sim \mathcal{A}$
return A_t

With probability $1 - \varepsilon$, the algorithm exploits. However, with probability ε, the algorithm explores. Note that the exploration does not exclude the greedy action.

1.3.1.2.3 Upper-Confidence-Bound Action Selection UCB action selection takes into account both how close the estimates of action values are to being maximal and the uncertainties in those estimates.

Algorithm 5: Upper-Confidence-Bound Action Selection

Input : $\mathcal{A}$ ..... set of all possible actions
t ..... time step
$Q_t(a)$ ..... estimation of value of action a on time step t
$N_t(a)$ ..... the number of times action a was selected prior to time t
c ..... degree of exploration

Output: A_t ..... action selection at time t

$A_t \leftarrow \arg\max_a \left\{ Q_t(a) + c\sqrt{\frac{\log t}{N_t(a)}} \right\}$
return A_t

The square root term can be seen as uncertainty term. Each time a is selected, the uncertainty term decreases. On the other hand, when a is not selected for a long time, the uncertainty increases.

The UCB action selection method cannot be extended to general non-stationary bandit problems, and has difficulty dealing with large state spaces.

1.3.1.3 Optimistic Initial Values

By simply setting the initial values $Q_0(\cdot)$ optimistically, we can encourage exploration easily. First, the algorithm sets initial values equally with high, optimistic values. Then the algorithm selects action, like normal methods. Since the values were highly optimistic, the rewards that the algorithm receives will likely be disappointing, encouraging other actions to be selected next time. The result is that all actions are tried several times before the value estimates converge.

This method can outperform ε-greedy method with realistic initial values when combined with greedy action selection method. However, it cannot be used in non-stationary problems, since its drive for exploration is effective only in the beginning steps.

1.3.2 Gradient Bandit Algorithm

Instead of estimating the action values and deciding which action to execute depending on the estimates, we can learn the *preference* of actions and use it to maximize the expected reward. This preference has no interpretation in terms of rewards; the higher the preference, the more often the action is selected. The preference is not absolute; it is relative values among the actions. If the preferences of actions are same, for example, 1000, the probability of each action being selected equals each other. We denote the preferences of action a in time step t as $H_t(a)$. The probabilities of choosing the action a at time step t is usually calculated using the *softmax* function, given as:

$$\mathbb{P}\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b\in\mathcal{A}} e^{H_t(b)}}, \tag{1.1}$$

and we denote it as $\pi_t(a)$.

1.3.2.1 Algorithm

Algorithm 6: Gradient Bandit Algorithm

Input : $\mathcal{A}$ ..... set of all possible actions
$\{H_0(a)\}_{a\in\mathcal{A}}$ ..... initial preferences
α ..... step size

Output: $\{H(a)\}$ ..... final preferences

$t \leftarrow 0$
$\bar{R}_t \leftarrow 0$
repeat
  foreach $a \in \mathcal{A}$ **do**
    $\pi_t(a) \leftarrow \frac{e^{H_t(a)}}{\sum_{b\in\mathcal{A}} e^{H_t(b)}}$
  $A_t \sim \text{Categorial}(\{\pi_t(a)\}_{a\in\mathcal{A}})$
  $R_t \leftarrow$ the given reward
  $\bar{R}_t \leftarrow$ the average (or weighted average for nonstationary) of rewards up through and including time t
  foreach $a \in \mathcal{A}$ **do**
    $H_{t+1}(a) \leftarrow H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{1}_{a=A_t} - \pi_t(a))$
  $t \leftarrow t + 1$
until convergence
return $\{H_t(a)\}_{a\in\mathcal{A}}$

Remark 1.3.2.1 (Gradient Bandit Algorithm as Stochastic Gradient Ascent)

The gradient bandit algorithm can be derived using stochastic apprximation to gradient ascent. First of all, our objective is to maximize the expected reward:

$$\mathbb{E}[R_t] = \sum_{a\in\mathcal{A}} \pi_t(a) q_*(a) \tag{1.2}$$

where $\pi_t(a)$ is given as 1.1. The update equation of exact gradient ascending would then be

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} \tag{1.3}$$

Taking a closer look at the update equation, we can get the following result;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left\{ \sum_{x \in \mathcal{A}} \pi_t(x) q_*(x) \right\} \tag{1.4}$$

$$= \sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} \tag{1.5}$$

$$= \sum_x (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} \tag{1.6}$$

where B_t is called *baseline*, and can be any value that does not depend on x. Note that the sum of probability is always 1, and this enables the equality of 1.6. Next we multiply equation by $1 = \frac{\pi_t(x)}{\pi_t(x)}$;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \sum_x \pi_t(x)(q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} / \pi_t(x) \tag{1.7}$$

$$= \mathbb{E}\left[(q_*(A_t) - B_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \tag{1.8}$$

$$= \mathbb{E}\left[(R_t - \bar{R}_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \tag{1.9}$$

where we have chosen the baseline $B_t = \bar{R}_t$ and substituted $q_*(A_t) = R_t$. Note that

$$\frac{\partial \pi_t(x)}{H_t(a)} = \frac{\partial}{\partial H_t(a)} \pi_t(x) \tag{1.10}$$

$$= \frac{\partial}{\partial H_t(a)} \left[\frac{e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} \right] \tag{1.11}$$

$$= \frac{\frac{\partial e^{H_t(x)}}{\partial H_t(a)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} \frac{\partial \sum_{b \in \mathcal{A}} e^{H_t(b)}}{\partial H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \tag{1.12}$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \tag{1.13}$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} - \frac{e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \tag{1.14}$$

$$= \mathbb{1}_{a=x} \pi_t(x) - \pi_t(x) \pi_t(a) \tag{1.15}$$

$$= \pi_t(x)(\mathbb{1}_{a=x} - \pi_t(a)). \tag{1.16}$$

Substituting this result to 1.9, we get

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E}\left[(R_t - \bar{R}_t) \pi_t(A_t)(\mathbb{1}_{a=A_t} - \pi_t(a)) / \pi_t(A_t) \right] \tag{1.17}$$

$$= \mathbb{E}[(R_t - \bar{R}_t)(\mathbb{1}_{a=A_t} - \pi_t(a))]. \tag{1.18}$$

Evaluating this expectation by sampling and substituting to 1.3 leads to

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{1}_{a=A_t} - \pi_t(a)) \tag{1.19}$$

Chapter 2

Markov Decision Processes

2.1 Introduction

In this chapter, we discuss **Markov decision process**, a framework that reinforcement learning heavily relys on. Markov decision process is the foundation of reinforcement learning algorithms. We mainly focus on discrete Markov decision process.

The following property of conditional expectation is used often in this document; for example in the derivation of Bellman equations.

$$\mathbb{E}[\mathbb{E}[X|Y,Z]|Y] = \mathbb{E}[X|Z] \tag{2.1}$$

2.2 Finite Markov Decision Process

2.2.1 Definition

The learner and decision maker is called *agent*, and the *environment* is the thing the agent interacts with. These two interact at each of a sequence of time step $t = 0, 1, 2, 3, \ldots$. At every time step t, the agent receives *state* $S_t \in \mathcal{S}$ of the environment and selects an action $A_t \in \mathcal{A}(s)$ based on that state s. Next step, the agent receives a reward $R_{t+1} \in \mathcal{R}$ and finds itself in a new state S_{t+1}. The MDP then give rise to the *trajectory* $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$.

The states, rewards and actions are all have finite number of elements in *finite* MDP. We will mainly discuss about finite MDP.

Definition 2.2.1.1 (Markov Decision Process (MDP))

Markov decision process is a tuple of five objects $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ where $\mathcal{S}$, $\mathcal{R}$, and $\mathcal{A}$ are sets of states, rewards, and actions. The function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is called *state-transition probability* and defined as

$$p(s'|s, a) = \mathbb{P}\{S_t = s'|S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r|s, a).$$

The function $p(s', r|s, a) = \mathbb{P}\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$ is called *dynamics* of the MDP. $\gamma \in [0, 1]$ is called *discount rate*, and is used to calculate the expected value of cummulative rewards, namely the *return* and the expectation of it is what the agent has to maximize. The return is defined as

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Note that the goal can be written in recursive manner; $G_t = R_{t+1} + \gamma G_{t+1}$.

2.2.2 Markov Property

The important property of Markov decision process is that R_t and S_t depends only on previous state and action; S_{t-1} and A_{t-1}. This is called **Markov property**. More formally, the Markov property can be defined as following;

Definition 2.2.2.1 (Markov Property)

Let S_t, R_t and A_t be the state, reward and action at time t. Then, the probability distribution satisfies **Markov property** if

$$P\{S_t = s', R_t = r|S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \ldots, S_0 = s_0, A_0 = a_0\} = P\{S_t = s', R_t = r|S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}\}$$

for all $s', s_i \in \mathcal{S}$, $r \in \mathcal{R}$ and $a_i \in \mathcal{A}$.

2.3 Episodic Task and Continuing Task

The *return* $G_t = R_{t+1} + \gamma^2 R_{t+2} + R_{t+3} + \ldots$, presented in Definition 2.2.1.1 shortly, is the value which we should maximize about expectation. The return may consist of finite rewards, or it can be formulated as an infinite sum.

When the agent-environment interaction can break naturally into subsequences, for example winning the chess game and starting it all again from the start, we call these subsequences **episodes**. All the episodes terminate in the special state called **terminal state**. Then, the return can be expressed in finite sum;

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-1} R_T$$

where the time of termination, T is a random variable that normally varies from episode to episode and $0 \leq \gamma \leq 1$ is the discount rate. The states other than teminal state is often written as $\mathcal{S}$, and all the states including terminal state is often written as $\mathcal{S}^+$.

When the agent-environment interaction cannot be expressed in terms of episodes, we call it **continuing task**. The

return now cannot be formulated as finite sum;

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.2}$$

$$= R_{t+1} + \gamma G_{t+1} \tag{2.3}$$

This equation still holds when the task is episodic; We can set $G_T = 0$ for terminal time T.

The episodic and continuing tasks can be expressed in unified notation by using special *absorbing state* which transitions only to itself and generates only zero rewards. Then the return can be expressed as

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

with possibility $T = \infty$ and $\gamma = 0$, but not both.

2.4 Policies and Value Functions

Now we introduce *value functions* which represents the estimate of *how good* it is for the agent to be in a given state or to perform a given action in a given state. The 'how good' is expressed in terms of expectations of returns.

2.4.1 Definitions

Definition 2.4.1.1 (Policy)

The **policy** $\pi(a|s)$ is a mapping from states to probablities of selecting each possible action, where $a \in \mathcal{A}(s)$.

If the agent is following policy $\pi(a|s)$ at time t, the probability of selecting $A_t = a$ in $S_t = s$ is $\pi(a|s)$.

Definition 2.4.1.2 (Value Function)

The **value function** $v_\pi(s)$ of a state s is the expected return when starting in s and follwing policy π, i.e.,

$$v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s\right]$$

where $\mathbb{E}_\pi$ denotes the expected value of a random variable given that the agent follows policy π. The function v_π itself is called **state-value function for policy** π.

The value of terminal state is always 0.

Definition 2.4.1.3 (Action-Value Function)

The **action-value function** $q_\pi(s, a)$ is the expected return starting from state s, taking the action a, and then following policy π.

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a\right]$$

Note that the value function and action-value function has following relationship:

Theorem 2.4.1.1 (Relationship of Value Function and Action-Value Function)

The state-value fuction $v_\pi(s)$ and action-value function $q_\pi(s,a)$ satisfies following property:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s,a)$$
$$q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a]$$

Proof. Use the equation 2.1. The second equation can be derived using equation 2.1 with the Markov property. Note that the expectation is taken about policies. □

These functions can be estimated from experience. This type of estimation method is one of Monte Carlo methods. When there are too much states and actions or when they are continuous, we can approximated the function using parametrized approximator, such as deep neural network.

2.4.2 Bellman Equation

The fundamental property of value functions that are used throughout reinforcement learning and dynamic programming is that they satisfy recursive relationship, called **Bellman equation** for v_π.

Theorem 2.4.2.1 (Bellman Expectation Equation for v_π)

Let v_π be a state-value function for policy π. Then the v_π satisfies following equation;

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] && (2.4)\\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] && (2.5)\\ &= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s'} \sum_{r} p(s', r|s, a) \big[r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] \big] && (2.6)\\ &= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_\pi(s')], \text{ for all } s \in \mathcal{S} && (2.7) \end{aligned}$$

which is called the **Bellman equation** for v_π.

The actions a are taken from $\mathcal{A}(s)$. The sum can be understood as expectation, since it is a sum over all three variables a, s, and r. It is just a sum of returns $r + \gamma v_\pi(s')$ multiplied by the probability $\pi(a|s)p(s', r|s, a)$.

The following is the Bellman equation for action-value function.

Theorem 2.4.2.2 (Bellman Expectation Equation for q_π)

Let q_π be an action-value function for policy π. Then q_π satisfies following equation;

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}_\pi[G_t|S_t=s,A_t=a] && (2.8)\\ &= \mathbb{E}_\pi[R_{t+1}+\gamma G_{t+1}|S_t=s,A_t=a] && (2.9)\\ &= \sum_{s',r} p(s',r|s,a)\big[r+\gamma\mathbb{E}_\pi[G_{t+1}|S_{t+1}=s']\big] && (2.10)\\ &= \sum_{s',r} p(s',r|s,a)\big[r+\gamma v_\pi(s')\big] && (2.11)\\ &= \sum_{s',r} p(s',r|s,a)\left[r+\gamma\sum_{a'\in\mathcal{A}(s')}\pi(a'|s')q_\pi(s',a')\right],\ \text{for all } s\in\mathcal{S}, a\in\mathcal{A}(s) && (2.12) \end{aligned}$$

which is called the **Bellman equation** for q_π.

Note that we used law of total expectation; check 2.1.

The Bellman expectation equations have unique solution. This can be proved by using the Banach fixed point theorem.

2.5 Optimal Policies and Optimal Value Functions

2.5.1 Definition

Value functions define a partial ordering over policies. We denote $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. The **optimal policy** is a policy that is better than or equal to all other policies. There is always at least one optimal policy. There can be more than one optimal policies, but they all share the same state-value function, namely the *optimal state-value function* and the same *optimal action-value function*.

Definition 2.5.1.1 (Optimal State-Value Function)

Let v_π be a state-value function. Then, the **optimal state-value function** v_* is defined as

$$v_*(s) = \max_\pi v_\pi(s)$$

for all $s \in \mathcal{S}$.

Definition 2.5.1.2 (Optimal Action-Value Function)

Let q_π be a action-value function. Then, the **optimal action-value function** q_* is defined as

$$q_*(a,s) = \max_\pi q_\pi(a,s)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

2.5.2 Bellman Optimality Equation

The optimal functions presented above also satisfies the Bellman equation, called the **Bellman optimality equation**;

Theorem 2.5.2.1 (Bellman Optimality Equation)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following **Bellman optimality equation**;

$$v_*(s) = \max_{a \in \mathcal{A}} q_*(s,a) \tag{2.13}$$

$$= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \tag{2.14}$$

$$= \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_*(s')] \tag{2.15}$$

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a] \tag{2.16}$$

$$= \sum_{s',r} p(s', r|s, a)[r + \gamma \max_{a'} q_*(s', a')] \tag{2.17}$$

Note that we used following relationship between optimal functions;

Theorem 2.5.2.2 (Relationship Between Optimal Functions)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following relationship;

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s,a) \tag{2.18}$$

Moreover, following holds for optimal policy π_*;

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}(s)} q_*(s,a) \tag{2.19}$$

Proof. The proof for first equation uses $a = b \Leftrightarrow a \leq b$ and $b \leq a$. The second equation can be proved using the first equation. □

Once one has v_*, determining an optimal policy is straightforward. We just have to make one step along, and then find out the actions that satisfy the Bellman optimality equation. We then create the policy that allocates positive probability only to these actions. This *greedy* selection seems that it ignores possibility of better alternatives in the long run. However, note that we are trying to maximize the *return*, which condsiders a long-run cummulative sum, and the optimal functions are derived using this return. Thus, we do not have to worry about the better alternatives.

Having q_*, it is even more strightforward to create an optimal policy. We don't even have to take a step ahead; since we can always find optimal action a, for any given state s.

The Bellman optimal equations have unique solution. This can be proved by using the Banach fixed point theorem.

Chapter 3

Dynamic Programming

3.1 Introduction

Dynamic programming (DP) refers to a collection of algorithms which finds out optimal policies given a *perfect* model of MDP (the dynamics of MDP $p(s', a|s, a)$.). Since it requires perfect MDP model and has expensive computational cost, DP is not used so much in real applications. However, the methods shown later (Monte Carlo, Temporal Difference) are all rooted from dynamic programming. DP is a foundation of these methods.

3.2 Policy Evaluation

First, we consider how to compute the state-value function v_π when a policy π is given. Using the Bellman equation, we can get an *iterative policy evaluation algorithm*. Starting with initial state-value function v_0, we update the state-value function as following;

$$v_{k+1}(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}|S_t = s)] \tag{3.1}$$

$$= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_k(s')] \tag{3.2}$$

for all $s \in \mathcal{S}$. We can see that $v_k = v_\pi$ is the fixed point for this update rule. The sequence $\{v_k\}$ is known to converge to v_π under the same condition that guarantee the existence of v_π. The algorithm pseudocode is given as follows.

Algorithm 7: Iterative Policy Evaluation Algorithm

Input : $p(s', r|s, a)$ ..... the dynamics of given finite MDP
π ..... given policy
Output: v_π ..... the state-value function

$\theta \leftarrow$ appropriate threshold value for convergence
initialize $V(s)$ for all $s \in \mathcal{S}^+$ except that $V(terminal) = 0$
repeat
  $\Delta \leftarrow 0$
  foreach $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$
return $V \simeq v_\pi$

One can notice that the algorithm 7 is not really same as the update equation 3.2. When updating the value of state, we may use the values of states that are newly updated, unlike the equation 3.2. However, this is also known to converge well, even more quickly than when only using the old values of states. This is intuitively reasonable since the new values can be regarded more closer to the real value.

3.3 Policy Improvement

Now we have evaluated the value function. Our next step is to improve the policy with the state-value function we computed. The improvement of the policy has its root on the *policy imporvement theorem*:

Theorem 3.3.0.1 (Policy Improvement Theorem)

Let π, v_π and q_π be a given deterministic policy and state-value function and action-value function of the policy π, respectively. Let π' be another deterministic policy. Then, the following holds:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \Rightarrow v_{\pi'}(s) \geq v_\pi(s) \text{ for all } s \in \mathcal{S} \tag{3.3}$$

Moreover, if there is a strict inequality of the equation 3.3 of left-hand side at any state, then there must be strict inequality of the right-hand side of the equation 3.3 at that state.

The idea behind the proof of this statement is quite straightforward. We let the policy π' to be identical to π except that

$\pi'(s) = a \neq \pi(s)$. It is given as follows:

$$\begin{aligned}
v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}|S_t = s, A_t = \pi'(s))] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})|S_t = s)] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma E_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2}|S_{t+1}, A_{t+1} = \pi'(S_{t+1}))|S_t = s]] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2})|S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3})|S_t = s] \\
&\vdots \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \ldots|S_t = s] \\
&= v_{\pi'}(s).
\end{aligned}$$

Now, considering the changes at all states and to all possible actions, we consider the new *greedy* policy π' given by

$$\pi'(s) = \arg\max_{a \in \mathcal{A}(s)} q_\pi(s, a) \tag{3.4}$$

$$= \arg\max_{a \in \mathcal{A}(s)} \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s, A_t = a] \tag{3.5}$$

$$= \arg\max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r|s, a)[r + \gamma v_\pi(s')] \tag{3.6}$$

This process, making the original policy greedy with respect to the value function of the original policy, is called *policy improvement*.

It is guranteed for a policy to strictly improve unless the original policy is already optimal; Let π' be a policy that is as good but not better than π. Then, $v_\pi = v_{\pi'}$ and from the equation 3.5, it follows that for all $s \in \mathcal{S}$,

$$\begin{aligned}
v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1})|S_t = s, A_t = a] \\
&= \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v_{\pi'}(s')].
\end{aligned}$$

And this is same as the Bellman optimal equation, and $v_{\pi'}$ must be v_*, the optimal value function, and so is v_π.

3.4 Obtaining Optimal Policy

3.4.1 Policy Iteration

Now, we can repeat the above two processes, policy evaluation and policy improvement, to obtain the optimal policy and optimal state-value function. Specifically, we start with initial policy π_0 and evaluate the state-value function v_{π_0}. Then we improve policy π_0 with v_{π_0} to get π_1, and evaluate next state-value function v_{π_1}. And we repeat it until we get π_* and v_*.

This way of finding optimal policy is called *policy iteration*.

Since the finite MDP has only finite number of policies, this process must converge to an optimal policy in finite number of iterations.

Algorithm 8: Policy Iteration (with iterative policy evaluation)

Input : $p(s', r|s, a)$ ..... the dynamics of given finite MDP
π ..... initial policy
v ..... initial state-value function

Output: π_* ..... optimal policy
v_* ..... optimal state-value function

$\theta \leftarrow$ appropriate threshold value for convergence
$\Delta \leftarrow 0$
policy-stable $\leftarrow$ false
repeat
  `// Policy Evaluation`
  repeat
    $\Delta \leftarrow 0$
    $V_{old}(s) \leftarrow V(s)$
    foreach $s \in \mathcal{S}$ **do**
      $v \leftarrow V(s)$
      $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
  until $\Delta < \theta$
  `// Policy Improvement`
  policy-stable $\leftarrow$ true
  foreach $s \in \mathcal{S}$ **do**
    $\pi'(s) \leftarrow \pi(s)$
    $\pi(s) \leftarrow \arg\max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
    if $\pi'(s) \neq \pi(s)$ **then**
      policy-stable $\leftarrow$ false
until *policy-stable* is true and $\max |V_{old}(s) - V(s)| < \theta$

3.4.2 Value Iteration

One drawback of policy iteration is that it involves policy evaluation, which may take a long time if the state space is too large. The good news is that the policy evaluation step in policy iteration can be truncated in several ways, without losing a convergence guarantees of policy iteration. One of such a way is *value iteration*. The value iteration combines policy improvement and truncated policy evaluation steps;

$$v_{k+1} = \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \tag{3.7}$$

$$= \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_k(s')] \tag{3.8}$$

The sequence $\{v_k\}$ is known to converge to v_*.

Algorithm 9: Value Iteration

Input : $p(s', r|s, a)$ ..... the dynamics of given finite MDP
$V(s)$ ..... initia value function, except that $V(terminal) = 0$

Output: π ..... final policy

$\theta \leftarrow$ appropriate threshold value
repeat
  $\Delta \leftarrow 0$
  foreach $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$
foreach $s \in \mathcal{S}$ **do**
  $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
return π

Faster convergence can often be achieved by interposing multiple policy evaluation sweeps between each policy improvement step.

3.5 Asynchronous Dynammic Programming

As one can see, the DP methods requires a sweep along the state space for oplicy evaluation. This can be expensive, if the given state space is too large.

Asynchronous DP algorithms do not sweep along the state space. The values are updated without any order in this algorithms, and even some states can be updated multiple times while other states are not. However, it cannot ignore the states; for the asynchronous DP algorithms to converge, they must visit all states.

The point is that we do not have to wait for policy evaluation step to finish. We can improve our policy without sweeping a state space for policy evaluation. We might even try to skip updating some states if it is unrelevant to optima behavior.

Asynchronous DP algorithms can easily be mixed up with real-time interaction.

3.6 Generalized Policy Iteration

Policy iteration involves two simultaneous interacting processes, the policy evaluation and policy improvement. In policy iteration method, these two processes wait each other to be finished. But, this is not neccessary, and the example of it is the value iteration. In asynchronous DP algorithms, these two processes intervene in mush finer manner. We use the term **general policy iteration** to denote the general idea of letting policy evaluation and improvement processes interact.

Chapter 4

Monte Carlo Methods

4.1 Introduction

In this chapter, the Monte Carlo methods are discussed. Unlike DP, Monte Carlo does not require a perfect MDP model. Monte Carlo learns from experience, or simulated experience, if a model is ready.

Monte Carlo methods are ways of solving reinforcement problem based on averaging sample returns. Unfortunately, the Monte Carlo method can be used only on episodic tasks.

4.2 Monte Carlo Prediction

When a policy is given, we can estimate a state-value function of the policy by **Monte Carlo prediction**. Monte Carlo prediction averages the sample value of desired states and use it as the estimate of state-value function. The averaging is done when each episode terminates, unlike ordinary DP method which updates in each step. This idea of sampling and averaging the returns underlies all Monte Carlo methods.

Precisely, Monte Carlo prediction are devided into two methods; *first-visit MC method* and *every-visit MC method*. The former one estimates $v_\pi(s)$ as the average of the returns first visits to s. The latter one averages the returns following all visits to s. These two MC methods are very similar but have slightly different theoretical properties.

Algorithm 10: First-Visit MC Prediction, for estimating v_π

Input : π ..... a given policy

Output: $V \simeq v_\pi$ ..... an estimate of v_π

```
// initialize
```

$V(s) \in \mathbb{R} \leftarrow$ arbitrary values, for all $s \in \mathcal{S}$

$Ret(s) \leftarrow$ empty list, for all $s \in \mathcal{S}$

repeat

  generate an episode following π: $S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$

  $G \leftarrow 0$

  foreach step of episode, $t = T-1, T-2, \ldots, 1, 0$ **do**

    $G \leftarrow \gamma G + R_{t+1}$

    `// in every-visit MC method, this if statement is ignored`

    if S_t did not appear in $S_0, S_1, S_2, \ldots, S_{t-1}$ **then**

      append G to $Ret(s)$

      $V(S_t) \leftarrow average(R(s))$

until convergence

Due to the law of large numbers, the first-visit MC method and every-visit MC method converges to $v_\pi(s)$ as the number of visits to s goes to infinity.

Note that the MC method does not bootstrap; DP method uses values of other states when estimating a value of a given state. MC does not. Due to this property of MC method, MC method is useful when one requires only values of a subset of states.

4.3 Monte Carlo Estimation of Action Values

If a model is not available, it is particularly useful to estimate action values rather than state values, since we do not know which action gives better return when a model is unavailable. The value we estimate now changes from $v_\pi(s)$ to $q_\pi(s, a)$. We just have to change state to state-action pair in algorithm 10.

The only complication is that many state-action pairs won't be visited. If π is a deterministic policy, we cannot get the values of action rather than $\pi(s)$. The problem of *exploiting and exploring* rises here again. One way to maintain the exploration is to make the episode start with desired state-action pair, and every state-action pair has nonzero probability of being selected. We call this the assumption of *exploring starts*. It is useful, but notice that it cannot be used in all situations; we cannot manually set the start when interacting with real-life environment.

4.4 Monte Carlo Control

After the evaluation of policy, we can improve our policy, as we did in DP. The difference is that we have action-value function $q_\pi(s, a)$ instead of $v_\pi(s)$. This means that we do not need model to figure out the greedy policy.

4.4.1 Policy Improvement

For policy improvement, we select a greedy policy with respect to current action-value function $q_\pi(s, a)$.

$$\pi'(s) = \arg\max_a q_\pi(s, a) \tag{4.1}$$

Note that this new policy π' satisfies the conditions of policy improvement theorem;

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \max_a q_\pi(s, a) && (4.2)\\ &\geq q_\pi(s, \pi(s)) && (4.3)\\ &\geq v_\pi(s). && (4.4) \end{aligned}$$

4.4.2 Algorithms

4.4.2.1 Monte Carlo Control with Exploring Start

Algorithm 11: Monte Carlo Control with Exploring Start

Input : π_0 ..... initial policy
Output: q_* ..... optimal action-value function
π_* ..... optimal policy

`// intialize`
$Q(s, a) \in \mathbb{R} \leftarrow$ arbitrary, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
$Ret(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$
repeat
  $S_0 \sim \mathcal{S}$, uniformly
  $A_0 \sim \mathcal{A}(S_0)$, uniformly
  Generate an episode starting from S_0, A_0, following π: $S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  foreach step of episode, $t = T-1, T-2, \ldots, 1, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    `// in every-visit MC method, this if statement is ignored`
    if the pair S_t, A_t did not appear in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$ **then**
      append G to $Ret(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow avg(Ret(S_t, A_t))$
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
until convergence
return Q, π

One can easily show that algorithm 11 converges to optimal policy. If it converges to suboptimal policy, the value function won't be optimal, and it will cause the policy to change.

4.4.2.2 Monte Carlo Control without Exploring Start

The following algorithm does not use the exploring start method. The algorithm resolves problem of maintaining exploration by using non-deterministic policy. In this algorithm, ϵ-greedy policy is used as the non-deterministic policy.

Before presenting the algorithm, we make a definition of ε-soft policy.

Definition 4.4.2.1 (ε-soft Policy)

The **ϵ-soft policy** π is a policy satisfying following condition;

$$\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

Algorithm 12: On-Policy First-Visit Monte Carlo Control

Input : $\epsilon > 0$ ..... small positive number for probability of choosing non-greedy action
π ..... initial ε-soft policy

Output: q_* ..... optimal action-value function
π ..... optimal policy

`// intialize`

$Q(s,a) \in \mathbb{R} \leftarrow$ arbitrary, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
$Ret(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$
repeat
  Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  foreach step of episode, $t = T-1, T-2, \ldots, 1, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    `// in every-visit MC method, this if statement is ignored`
    if the pair S_t, A_t did not appear in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$ **then**
      append G to $Ret(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow avg(Ret(S_t, A_t))$
      $A_* \leftarrow \arg\max_a Q(S_t, a)$
      $\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(S_t)|} & \text{if } a = A_* \\ \frac{\epsilon}{|\mathcal{A}(S_t)|} & \text{if } a \neq A_* \end{cases}$
until convergence
return Q, π

We can show that this algorithm works well by showing that ϵ-greedy algorithm satisfies conditions of policy improvement theorem over all ε-soft policies;

Proof. Let π be any ε-soft policy. Let π' be ϵ-greedy policy. Then,

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a|s) q_\pi(s, a) \tag{4.5}$$

$$= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \max_a q_\pi(s, a) \tag{4.6}$$

$$\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1 - \epsilon} q_\pi(s, a) \tag{4.7}$$

$$= v_\pi(s) \tag{4.8}$$

□

Note that we used that the maximum is always bigger than weighted average with weights that sum to 1 in line 4.7. The equality can hold only when π' and π are both optimal policy among the ε-soft policies. This can be proved by using the uniqueness of the solution of Bellman optimal equation of state-value function.

4.4.3 Dealing with Infinite Iterations

The algorithms shown earlier assumes that the loop is repeated infinitely. However, it is impossible to repeat forever in real world. Two methods can be used to relax the infinite loop condition.

1. Hold the idea of approximating q_π; This idea was used along almost all algorithms we saw.
2. Give up trying to approximate q_π; This is the idea of GPI. One extreme example of this method is value iteration of DP method. We may not get the precise state value in certain time step, but the final result is still optimal.

4.5 Off-policy Monte Carlo Method

4.5.1 Introduction

The dilemma of reinforcement learning is always this: exploit or explore. One of the way of solving this dilemma is to use **off-policy** method.

Off-policy method uses two policy, one which is learned about and becomes optimal policy and one that is more exploratory and us used to generate behavior. The policy being learned is called **target policy**, and the policy used to generate behavior is called **behavior policy**.

To use episodes generated by behavior policy b to estimate values for target policy π, following condition must be satisfied.

Definition 4.5.1.1 (Assumption of Coverage)

Let π be a target policy and b a behavior policy. Then, the following **assumption of coverage** must be satisfied.

$$\pi(a|s) > 0 \Rightarrow b(a|s) > 0 \tag{4.9}$$

That is, we require that every action taken under π is also taken, at least occasionally, under b.

4.5.2 Importance Sampling

Almost all off-policy method uses **importance sampling** method. A general introduction to importance sampling method is given in this document. Suppose that we generated an episode starting with state S_t, and got the state-action trajectory $A_t, S_{t+1}, A_{t+1}, \ldots, S_T$. Then, the probability of the trajectory occuring under policy π is

$$\begin{aligned}
&\mathbb{P}\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T | S_t, A_{t:T-1} \sim \pi\} && (4.10)\\
&\quad = \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1}) && (4.11)\\
&\quad = \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k) && (4.12)
\end{aligned}$$

Similarly, the probabilty of the trajectory occuring under policy b is $\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)$. Then, the **importance-sampling ratio** $\rho_{t:T-1}$ is defined as following;

Definition 4.5.2.1 (Importance-sampling Ratio of Target and Behavior Policy)

Let π and b be the target and behavior policy, respectively. Then, the **importance-sampling ratio** $\rho_{t:T-1}$ is defined as

$$\rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)} \tag{4.13}$$

Then, the expected return (values) are calculated by multiplying the $\rho_{t:T-1}$ to the returns G_t due to the policy b.

$$\mathbb{E}[\rho_{t:T-1} G_t | S_t = s] = v_\pi(s) \tag{4.14}$$

4.5.3 Off-policy Monte Carlo Prediction

Now we apply the off-policy method to Monte Carlo prediction. We define some notations here.

- $\mathfrak{T}(s)$: the set of all time steps in which state s is visited.
- $T(t)$: the first time of termination following time t.

Using these notations, $\{G_t\}_{t \in \mathfrak{T}(s)}$ are the returns that pertain to state s and $\{\rho_{t:T(t)-1}\}_{t \in \mathfrak{T}(s)}$ are the corresponding importance-sampling ratios.

There are two ways of estimating the value. First is the **ordinary importance sampling** and the second is **weighted importance sampling**. The first method estimates value as

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathfrak{T}(s)|}. \tag{4.15}$$

The second method estimates value as

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathfrak{T}(s)} \rho_{t:T(t)-1}}. \tag{4.16}$$

The ordinary estimator is always v_π in the expectation (unbiased) but can be extreme, and the weighted estimator may produce v_b in the expectation (when only a one sample is observed) but the variance is bounded.

Following is the table of biasedness and unboundedness of variance of each method.

	ordinary	weighted
first-visit	unbiased, unbounded	biased, bounded
every-visit	biased, unbounded	biased, bounded

Since the states in every-visit method is correlated, the estimator is biased, but is asymptotically unbiased as the number of sample grows. In practice, the weighted estimator usually has dramatically lower variance and is strongly preferred.

4.5.3.1 Incremental Implementation

Incremental implementation can be used in off-policy Monte Carlo Prediction as we did in bandit problem. The difference is that the update occurs episode by episode, not step by step.

The incremental implementation of ordinary importance sampling is simple, since we just have to replace the reward to the weighted return of the algorithm 1.

Weighted importance sampling can also be implemented incrementally. Let $G_1, G_2, \ldots, G_{n-1}$ be a sequence of returns, all starting in same state s, each with a corresponding weight $W_i = \rho_{t_i:T(t_i)-1}$ where t_i denotes the time when state s occured, with return G_i. Let the weighted estimate of value of s be $V_n(s) = \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}$. The update rule for $V_n(s)$ is

$$V_{n+1}(s) = V_n(s) + \frac{W_n}{C_n}\Big[G_n - V_n\Big] \tag{4.17}$$

where

$$C_{n+1} = C_n + W_{n+1}, \tag{4.18}$$

with $C_0 = 0$.

Algorithm 13: Off-policy Monte Carlo Prediction

Input : π ..... target policy

Output: q_π ..... action-value function of policy π

```
// initialize
foreach (s, a) where s ∈ S, a ∈ A(s) do
    Q(s, a) ∈ ℝ ← arbitrary
    C(s, a) ← 0
repeat
    b ← any policy satisfying assumption of coverage with π
    Generate an episode following b: S_0, A_0, R_1, ..., S_{T-1}, A_{T-1}, R_T
    G ← 0
    W ← 1
    for t ∈ T − 1, T − 2, ..., 0, while W ≠ 0 do
        G ← γG + R_{t+1}
        C(S_t, A_t) ← C(S_t, A_t) + W
        Q(S_t, A_t) ← Q(S_t, A_t) + (W / C(S_t, A_t)) [G − Q(S_t, A_t)]
        W ← W π(A_t|S_t) / b(A_t|S_t)
until convergence
return Q
```

4.5.4 Off-policy Monte Carlo Control

Now we present the off-policy Monte Carlo control method. The target policy π and behavior policy b are used, and the assumption of coverage is satisfied. To assure that every possible state-action pair is visited, we set b to be an ε-soft policy. π is a deterministic policy.

Algorithm 14: Off-policy Monte Carlo Control

Input : .....

Output: q_* ..... optimal action-value function

π_* ..... optimal policy

// intialize

foreach (s, a) where $s \in \mathcal{S}, a \in \mathcal{A}(s)$ **do**
  $Q(s, a) \in \mathbb{R} \leftarrow$ arbitrary
  $C(s, a) \leftarrow 0$
  $\pi(s) \leftarrow \arg\max_a Q(s, a)$
repeat
  $b \leftarrow$ any ε-soft policy
  Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  for $t \in T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    if $A_t \neq \pi(S_t)$ **then**
      exit inner loop (proceed to next episode)
      // since the policy pi is deterministic, pi(S)!=A leads to zero weight.
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$
until convergence
return $Q \simeq q_*$, $\pi \simeq \pi_*$

Note that since π is deterministic, the weight W is zero if $A_t \neq \pi(S_t)$ in any time step t. Multiplying $\frac{1}{b(A_t|S_t)}$ instead of $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$ will seem reasonable now.

4.6 Reducing Variance of Estimators

4.6.1 Discount-aware Importance Sampling

The off-policy method we dicussed earlier uses importance-sampling, without taking into account the internal structure of return. **Discount-aware importance sampling** uses this structure to reduce the variance of off-policy estimators.

The idea is to think of γ, the discount rate, as the probability of termination. We define *flat partial returns* $\bar{G}_{t:h} =$

$R_{t+1} + R_{t+2} + \cdots + R_h$ where $0 \leq t < h \leq T$. h is called *horizon*. Then, the conventional return can be written as

$$
\begin{aligned}
G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T && (4.19)\\
&= (1-\gamma) R_{t+1} && (4.20)\\
&+ (1-\gamma)\gamma (R_{t+1} + R_{t+2}) && (4.21)\\
&+ (1-\gamma)\gamma^2 (R_{t+1} + R_{t+2} + R_{t+3}) && (4.22)\\
&\vdots && (4.23)\\
&+ (1-\gamma)\gamma^{T-t-2} (R_{t+1} + R_{t+2} + \cdots + R_{T-1}) && (4.24)\\
&+ \gamma^{T-t-1} (R_{t+1} + R_{t+2} + \cdots + R_T) && (4.25)\\
&= (1-\gamma) \sum_{h=t+1}^{T-1} \gamma^{h-t-1} \bar{G}_{t:h} + \gamma^{T-t-1} \bar{G}_{t:h} && (4.26)
\end{aligned}
$$

Now, we define two estimators analogous to ordinary and weighted importance-sampling estimators, with importance-sampling ratio truncated similarly.

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \left((1-\gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} \bar{G}_{t:h} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \bar{G}_{t:T(t)} \right)}{|\mathfrak{T}(s)|} \tag{4.27}$$

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \left((1-\gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} \bar{G}_{t:h} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \bar{G}_{t:T(t)} \right)}{\sum_{t \in \mathfrak{T}(s)} \left((1-\gamma) \sum_{h=t+1}^{T(t)-1} \gamma^{h-t-1} \rho_{t:h-1} + \gamma^{T(t)-t-1} \rho_{t:T(t)-1} \right)} \tag{4.28}$$

We call these two estimators **discounting-aware importance sampling estimators**.

4.6.2 Per-decision Importance Sampling

Lets inspect the weighted return $\rho_{t:T-1} G_t$. It can be writte as the sum of rewards multiplied by discount rate

$$\rho_{t:T-1} G_t = \rho_{t:T-1} R_{t+1} + \gamma \rho_{t:T-1} R_{t+2} + \gamma^2 \rho_{t:T-1} R_{t+3} + \cdots + \gamma^{T-t-1} \rho_{t:T-1} R_T \tag{4.29}$$

Note that the first term can be written as $\rho_{t:T-1} R_{t+1} = \frac{\pi(A_t|S_t) \cdots \pi(A_{T-1}|S_{T-1})}{b(A_t|S_t) \cdots b(A_{T-1}|S_{T-1})} R_{t+1}$. The expectations of these factors are all 1.

$$\mathbb{E}\left[\frac{\pi(A_k|S_k)}{b(S_k|A_k)} \right] = 1. \tag{4.30}$$

With more steps, one can show that

$$\mathbb{E}[\rho_{t:T-1} R_{t+k}] = \mathbb{E}[\rho_{t:t+k-1} R_{t+k}]. \tag{4.31}$$

Then defining $\tilde{G}_t = \rho_{t:t} R_{t+1} + \gamma \rho_{t:t+1} R_{t+2} + \gamma^2 \rho_{t:t+2} R_{t+3} + \cdots + \gamma^{T-t-1} \rho_{t:T-1} R_T$, it follows that

$$\mathbb{E}[\rho_{t:T-1} G_t] = \mathbb{E}[\tilde{G}_t]. \tag{4.32}$$

We call this **per-decision importance sampling**. The estimator analogous to the ordinary importance sampling estimator is

$$V(s) = \frac{\sum_{t \in \mathfrak{T}(s)} \tilde{G}_t}{|\mathfrak{T}(s)|} \tag{4.33}$$

which we expect to sometime be of lower variance. There is no obvious per-decision version of weighted importance sampling. The estimators proposed up to today are all non-consistent. (They do not converge to true value even with infinite data.)